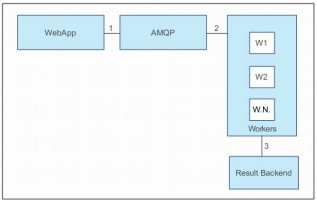

Figure 1: Block diagram of Celery architecture

```
from celery import Celery

#Configure celery.
celery = Celery('tasks', broker='amqp://guest@localhost//')

@celery.task    #Decorator which defines the underlying
function as a celery task.
def fetch_data(json_name):
        sleep(10)
        url_to_open = "http://localhost/%s" % json_name
        req = urllib2.Request(url_to_open)
        opener = urllib2.build_opener()
        f = opener.open(req)
        data_fetched = simplejson.load(f)
        print data_fetched
        return data_fetched
```

Now run the celery daemon from the terminal using the following command:

```
celery worker -A tasks --loglevel=INFO
```

These are the minimum arguments you need to pass to start the service. Other options like events, concurrency levels and CeleryBeat can also be passed as arguments. You'll learn about them later in the article.

In another terminal, use the Python interpreter to call the tasks module file:

```
>>> from tasks import add, fetch_data
>>> result = fetch_data.delay('sample.json')
```

Next, track the task state/fetch the result. There are a variety of ways to achieve this, depending on your use case. For example:

- You just want to execute the task, and don't want to save the result.
- You might want to check if the task has finished executing, or is still pending.
- Do you want to save the result in the message queue itself, or in MySQL or a back-end of your choice?

To achieve the third, you need to configure this setting in your *tasks.py* file, as follows:

```
celery = Celery('tasks', backend='amqp', broker='amqp://
guest@localhost//')
```

Now the message queue is configured to save the result of the job. You can configure any back-end that you wish to use here. This is how an immediate task is executed, though there might be use-cases wherein you would want to run scheduled jobs. To run a task as a scheduled task, you need to define the schedule in the decorator of the task, as follows:

```
@periodic_task(run_every=datetime.timedelta(minutes=1))
def print_name():
        print "Welcome to Tutorial"
```

The entry for period can be in the form of timedelta or in the form of *cron* too. Now the command for running the daemon would be:

```
celery worker -A tasks --loglevel=INFO -B # Where B means
running
```

CeleryBeat is used for periodic tasks; if the argument *–B* is not passed, then it will not run periodic tasks.

Next, let's look at how we can integrate Celery with Web frameworks—in this case, Django.

Integrating Celery with Django

Create a Django project using *django-admin.py startproject simple_django_project*, and then create an app in the project with *python manage.py start app celery_demo*. Next, install *django-celery* with *pip install django-celery* and then modify the *settings.py* file to configure the message queue, as shown below:

```
import djcelery
djcelery.setup_loader ()

INSTALLED_APPS = (
    ...
    'Djcelery' ,
)

BROKER_HOST = "localhost"
BROKER_PORT = 5672
BROKER_USER = "myusername"
BROKER_PASSWORD = "mypassword"
BROKER_VHOST = "myvhost"
```

Next, sync the database with *python manage.py syncdb*, after which create *tasks.py* inside the app. Now you can create

Continued on page 60...